



Daffodil International University

Lab Final Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	40
Understanding/Analysis	10					
Implementation	15					
Accuracy	10					
Task Efficiency	5					
Total obtained mark						
Comments						

Semester: Summer-2026

Student Name: Md. Abdullah Al Noman

Student ID: 232-35-003

Batch: 41

Section: G1

Course Code: DS 332

Course Name: Introduction to

Data Science and Data Management & Analysis Lab

Course Teacher Name: Syed Shams Islam

Designation: Lecturer

Submission Date: 06 / 08 / 2026

Contents

Notebook Practice Solution

1. Notebook 1: 5 Exercises
2. Notebook 2: 5 Exercises
3. Notebook 3: 5 Exercises
4. Notebook 4: 5 Exercises

Project Modeling:

1. Feature Engineering
2. Modeling
3. Description of Algorithms used
4. Results with Appropriate metrics
5. Discussion and conclusion

Notebook Problems and Solution

Notebook 1

Exercise 1. How many rows and columns does tips have? Which columns are numeric and which are categorical? Are any values missing?

```
# Exercise 1
print("Rows and Columns:", tips.shape)
tips.info()
```

[28] ✓ 0.0s Python

... Rows and Columns: (244, 7)
<class 'pandas.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
Column Non-Null Count Dtype

0 total_bill 244 non-null float64
1 tip 244 non-null float64
2 sex 244 non-null category
3 smoker 244 non-null category
4 day 244 non-null category
5 time 244 non-null category
6 size 244 non-null int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB

No values are missing, and total bill, tip, size are numerical and others are categorical

Exercise 2. Show summary statistics for the numeric columns. What is the largest total bill?

```
# Exercise 2

print("Largest Total Bill:", tips['total_bill'].max())
tips.describe()
```

[33] ✓ 0.0s Python

... Largest Total Bill: 50.81

...

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

Largest total bill is 50.81

Exercise 3. How many smokers vs. non-smokers are in the data? (Hint: value_counts.)

```
# Exercise 3
tips['smoker'].value_counts()
```

[36] ✓ 0.0s Python

```
...
smoker
No      151
Yes      93
Name: count, dtype: int64
```

Exercise 4. Select all rows where the bill is above 40 dollars and the party size is at least 4. How many are there?

```
# Exercise 4
data = tips[(tips['total_bill'] > 40) & (tips['size'] > 4)]
print(data)
```

[39] ✓ 0.0s Python

```
...
   total_bill  tip  sex smoker  day  time  size
142    41.19  5.0  Male    No  Thur  Lunch    5
156    48.17  5.0  Male    No   Sun  Dinner    6
```

Exercise 5. Using groupby, compute the average tip for each day of the week. Which day tips best?

```
# Exercise 5
mean_tip = tips.groupby('day')['tip'].mean().round(2)
print(mean_tip)
print("Best Day: ", mean_tip.idxmax())
```

[43] ✓ 0.0s Python

```
...
day
Thur    2.77
Fri     2.73
Sat     2.99
Sun     3.26
Name: tip, dtype: float64
Best Day: Sun
```

Best day is Sunday

Notebook 2

Exercise 1. Standardize the column names: strip spaces, lowercase, and replace (\$) so the names are valid identifiers like salary.

```
# Exercise 1
messy.columns = messy.columns.str.strip().str.lower().str.replace('($)', '')
messy.columns

... Index(['name', 'age', 'city', 'salary'], dtype='object')
```

Exercise 2. Age is stored as text and contains an impossible value. Convert it to numeric and deal with the 199. Justify your choice.

```
# Exercise 2
messy['age'] = pd.to_numeric(messy['age'], errors = 'coerce')
messy.loc[messy['age'] > 120, 'age'] = np.nan
age_median = messy['age'].median()
messy['age'] = messy['age'].fillna(age_median)
messy['age']

...
   age
0  25.0
1  32.0
2  32.0
3  32.0
4  28.0
5  32.0
6  41.0

dtype: float64
```

After converting to numeric, I have replaced the age with the median value of the dataset, cause 199 age is nearly impossible as an age, and the reason behind replacing with median cause the median will less affect the distribution.

Exercise 3. Clean the Name and City text (strip, consistent case), then find and remove the truly duplicated row.

```
messy_duplicate = messy.copy()
messy_duplicate['name'] = messy_duplicate['name'].str.strip().str.title()
messy_duplicate['city'] = messy_duplicate['city'].str.strip().str.title()
messy_duplicate = messy_duplicate.drop_duplicates()
messy_duplicate['city'] = messy_duplicate['city'].fillna(messy_duplicate['city'].mode()[0])
messy_duplicate

...
   name  age  city  salary
0  Alice  25.0  Dhaka  50,000
1   Bob  32.0  Dhaka  62,000
2   Bob  32.0  Chittagong  62,000
3  Charlie  32.0  Dhaka  58,000
4   Dana  28.0  Dhaka   n/a
6   None  41.0  Sylhet  71,000
```

Exercise 4. Convert Salary(\$) to numeric (careful: commas and "n/a"), then fill the missing salary with a value you think is reasonable and write one sentence defending the choice.

```
# Exercise 4
messy_duplicate['salary'] = messy_duplicate['salary'].str.replace(',', '').str.replace('n/a', '')
messy_duplicate['salary'] = pd.to_numeric(messy_duplicate['salary'], errors='coerce')
messy_duplicate['salary'] = messy_duplicate['salary'].fillna(messy_duplicate['salary'].median())
messy_duplicate
```

	name	age	city	salary
0	Alice	25.0	Dhaka	50000.0
1	Bob	32.0	Dhaka	62000.0
2	Bob	32.0	Chittagong	62000.0
3	Charlie	32.0	Dhaka	58000.0
4	Dana	28.0	Dhaka	62000.0
6	None	41.0	Sylhet	71000.0

Next steps: [Generate code with messy_duplicate](#) [New interactive sheet](#)

I have filled the Salary with median cause the salary median will less effect on the total distribution and data will not skewed by extreme values.

Exercise 5. Save your cleaned DataFrame to messy_clean.csv and reload it to verify.

```
# Exercise 5

messy_duplicate.to_csv('messy_clean.csv', index = False)
check = pd.read_csv('messy_clean.csv')
print("saved and reloaded:", check.shape)
check.head()
```

saved and reloaded: (6, 4)

	name	age	city	salary
0	Alice	25.0	Dhaka	50000.0
1	Bob	32.0	Dhaka	62000.0
2	Bob	32.0	Chittagong	62000.0
3	Charlie	32.0	Dhaka	58000.0
4	Dana	28.0	Dhaka	62000.0

Notebook 3

Exercise 1. Create `total_cost = fare + tip + tolls`, then use `pd.qcut` to bin it into "cheap", "normal", "expensive" (3 equal-sized groups). How many rides fall in each?

```
# Exercise 1
taxis['total_cost'] = taxis['fare'] + taxis['tip'] + taxis['tolls']
taxis['cost_level'] = pd.qcut(taxis['total_cost'], q=3, labels=['cheap', 'normal', 'expensive'])
taxis['cost_level'].value_counts()
```

cost_level	count
cheap	2170
normal	2141
expensive	2122

dtype: int64

Exercise 2. Create `speed_mph = distance / (duration_min / 60)`. Check it with `.describe()` do you find suspicious values? Where do they come from?

```
# Exercise 2
taxis['speed_mph'] = taxis['distance'] / (taxis['duration_min'] / 60)
taxis['speed_mph'].describe()
```

```
***
      speed_mph
count  6427.000000
mean    13.090678
std     68.040163
min      0.000000
25%     7.567081
50%    10.042553
75%    13.709845
max   4834.285714
dtype: float64
```

Suspicious values comes from Min and max, because speed divided by 0 means infinity that's why its 0, and max value extremely high because the entry maybe high.

Exercise 3. Using `map`, create a `payment_short` column where "credit card" → "card" and "cash" stays "cash".

```
# Exercise 3
taxis.columns
taxis['payment_short'] = taxis['payment'].map({'credit card': 'card', 'cash': 'cash'})
taxis['payment_short'].value_counts()
```

```
***
      count
payment_short
card      4577
cash     1812
dtype: int64
```

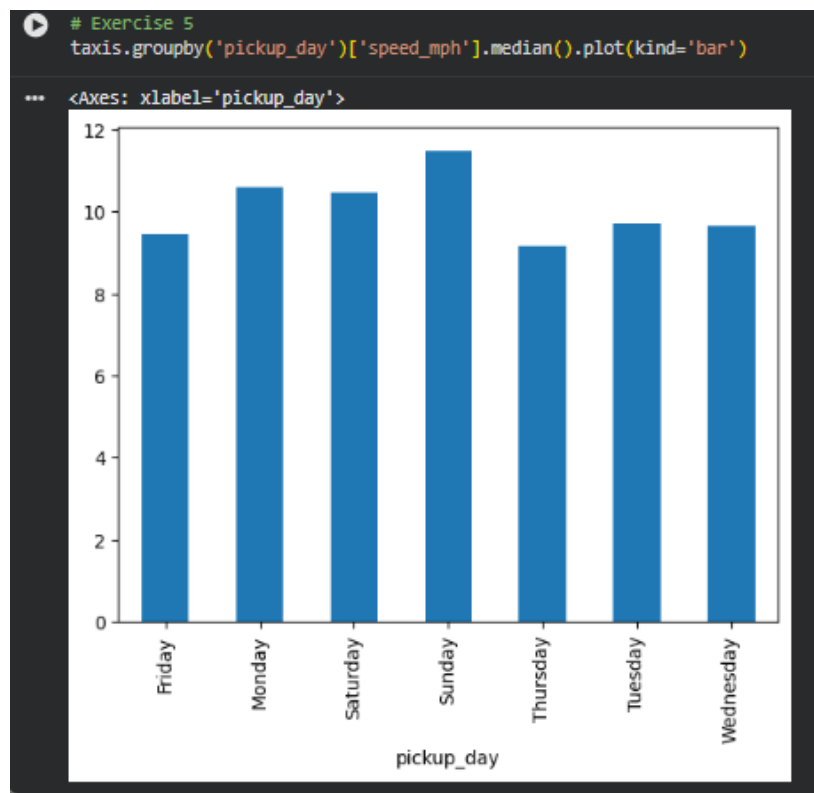
Exercise 4. Using `groupby().transform()`, add the average tip per `pickup_day`, then find which day has the most generous average tip.

```
# Exercise 4
taxi['pickup_day'] = taxi['pickup'].dt.day_name()
taxi['avg_tip_by_day'] = taxi.groupby('pickup_day')['tip'].transform('mean')
taxi.groupby('pickup_day')['tip'].mean().sort_values(ascending=False)
```

	tip
Wednesday	2.087329
Thursday	2.062177
Monday	2.027316
Tuesday	2.001370
Friday	1.980108
Sunday	1.908710
Saturday	1.815143

dtype: float64

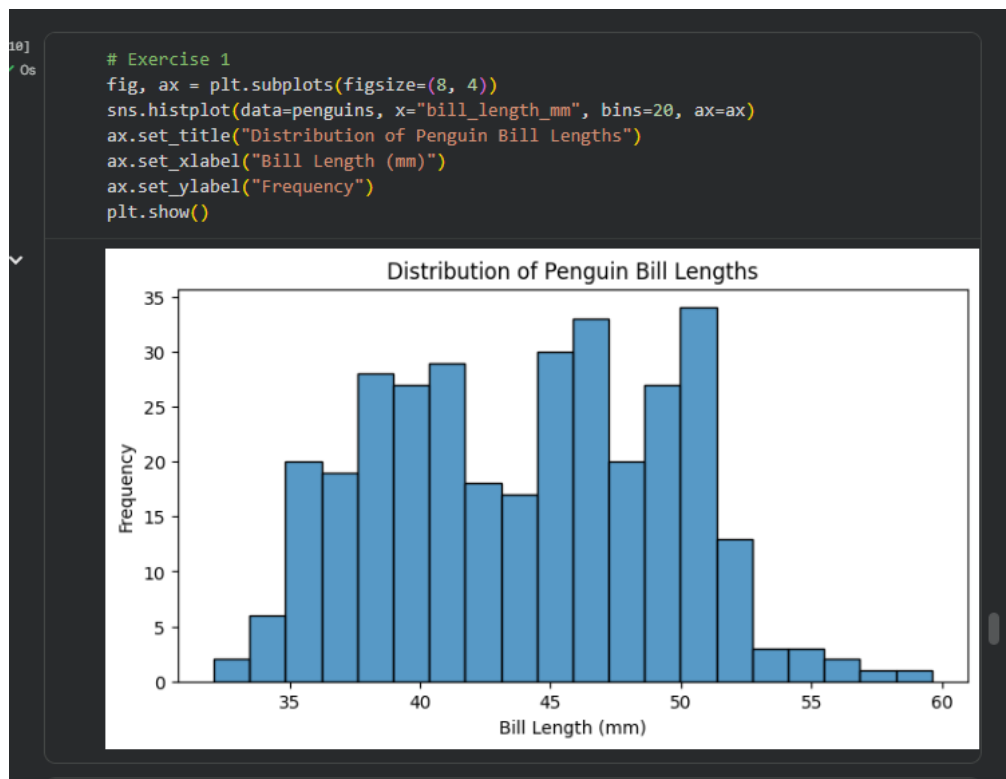
Exercise 5. Validate one of your new features with a quick plot (e.g. a bar chart of average speed per pickup hour). Write one sentence about what it shows.



The bar chart shows that the speed mile per hour on specific days, and it also shows that the speed changes on certain days based on the median value

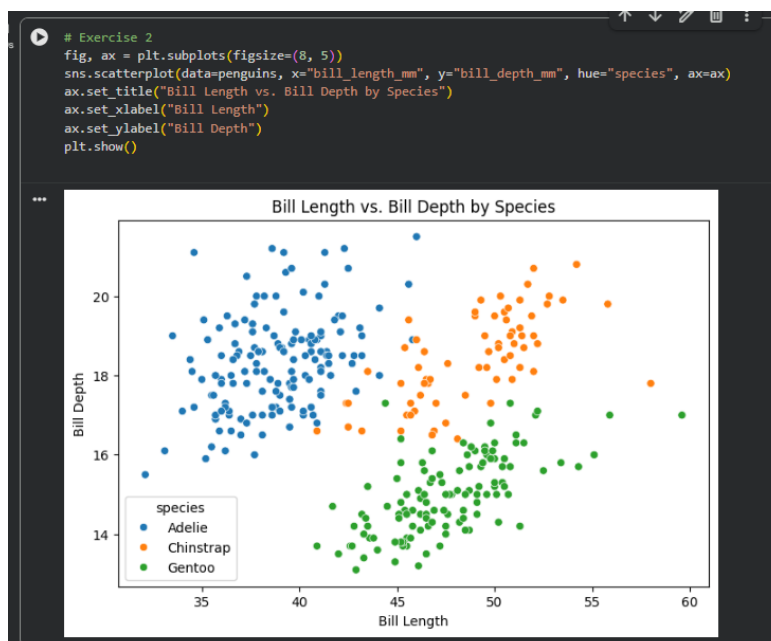
Notebook 4

Exercise 1. A histogram of `bill_length_mm` with a sensible number of bins. Is the distribution skewed? Could there be more than one bump and why might that be?



The Distribution has isn't perfect bell curve slightly right skewed, it has multiple bumps because this dataset contains three different species of penguins and each has its own average bill length

Exercise 2. A scatter plot of `bill_length_mm` vs `bill_depth_mm`, colored by species. Do clusters appear?

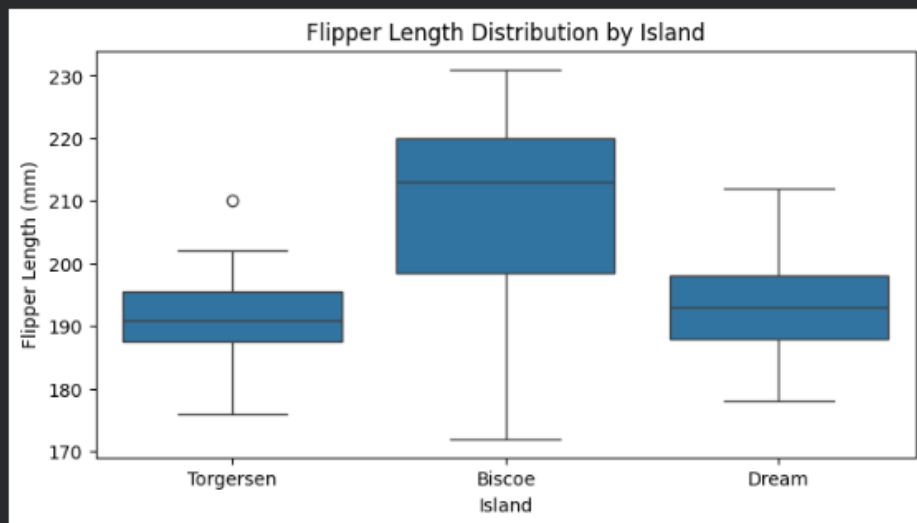


Exercise 3. Grouped boxplots of flipper_length_mm per island. Which island hosts the biggest penguins? (Bonus: which species lives there? Check with value_counts.)

```
# Exercise 3

fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=penguins, x="island", y="flipper_length_mm", ax=ax)
ax.set_title("Flipper Length Distribution by Island")
ax.set_xlabel("Island")
ax.set_ylabel("Flipper Length (mm)")
plt.show()

biscoe_penguins = penguins[penguins["island"] == "Biscoe"]
print(biscoe_penguins["species"].value_counts())
```



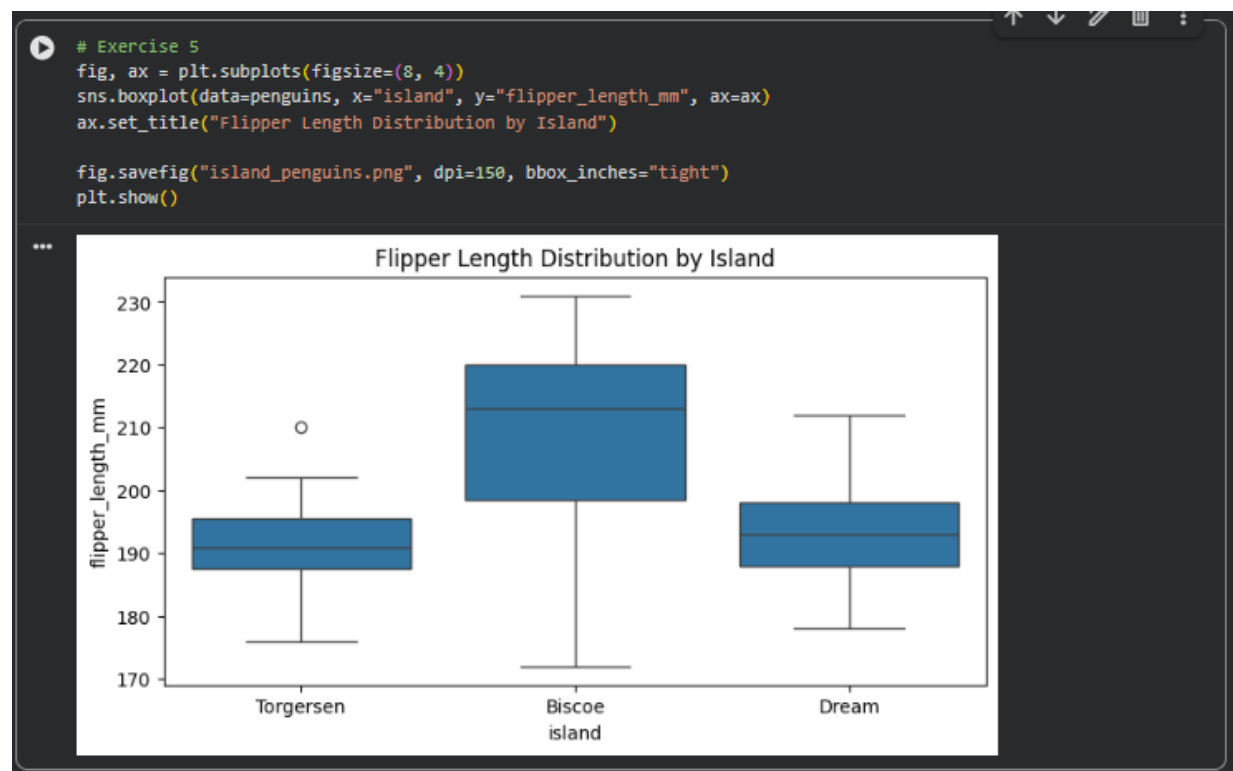
```
species
Gentoo    119
Adelie     44
Name: count, dtype: int64
```

Biscoe island hosts the penguins with largest flipper lengths, and gentoo species lives on Biscoe

Exercise 4. A figure with two subplots side by side: left, a countplot of species; right, a barplot of mean body_mass_g per species. Use `plt.subplots(1, 2, ...)`.



Exercise 5. Save your favorite of the four as a PNG at `dpi=150` and download it from the Colab file panel.



Modeling

Feature Engineering

This step exists because raw columns are rarely the best inputs to a model. Feature engineering is where we turn "what the data literally recorded" into "what actually helps a model learn a pattern." Three kinds of work happen here:

1. **Derived features:** ratios/rates that capture a relationship, not just a raw count
2. **Encoding:** models need numbers, not text, so `continent` (a category) has to become numeric
3. **Handling missing values:** EDA-stage NaN handling was about honesty. Modeling-stage NaN handling is stricter: models literally cannot consume NaN at all, so every remaining NaN must be dropped or imputed before this goes near `.fit()`.

Derived features:

Feature	Formula	Better then beacuse
fatality_rate	$\text{total_deaths} / \text{total_confirmed}$	Raw death counts mix up 'bad outbreak' with 'big population.' This isolates how deadly the disease was, given someone already got infected.
recovery_rate	$\text{total_recovered} / \text{total_confirmed}$	Shows what share of confirmed cases recovered, independent of country size.
testing_intensity	$\text{total_tests} / \text{population}$	Same idea as total_tests_per_1m, just scaled 0–1 instead of per-million easier for a model to work with, same information.

```
df_m = df.copy()

# Derived ratio features - guard against divide-by-zero with .replace
df_m['fatality_rate'] = (df_m['total_deaths'] / df_m['total_confirmed']).replace(0, np.nan)
df_m['recovery_rate'] = (df_m['total_recovered'] / df_m['total_confirmed']).replace(0, np.nan)
df_m['testing_intensity'] = (df_m['total_tests'] / df_m['population'])

df_m[['country', 'fatality_rate', 'recovery_rate', 'testing_intensity']].head()
```

0.0s Python

	country	fatality_rate	recovery_rate	testing_intensity
0	Afghanistan	0.042897	0.904807	0.023455
1	Albania	0.012690	0.986399	0.632857
2	Algeria	0.025864	0.671032	0.005093
3	Andorra	0.003629	0.973076	3.223924
4	Angola	0.019154	0.979384	0.043136

Encoding the continent column:

continent is a category with no natural order Asia isn't 'more' than Europe so one-hot encoding was used (one 0/1 column per continent) instead of assigning arbitrary numbers. Numbers would falsely tell the model that continents have a ranking or a measurable distance between

them, which isn't true. With `drop_first=True`, only 5 dummy columns were kept out of 6 continents (Asia, Australia/Oceania, Europe, North America, South America), since the 6th (Africa) is implied when all others are 0. This also avoids multicollinearity, which specifically breaks linear regression's math.

```
continent_dummies = pd.get_dummies(df_m['continent'], prefix='continent',
drop_first=True)
df_m = pd.concat([df_m, continent_dummies], axis=1)
continent_dummies.columns.tolist()

✓ 0.0s Python
```

```
['continent_Asia',
'continent_Australia/Oceania',
'continent_Europe',
'continent_North America',
'continent_South America']
```

Handling missing values for modeling

EDA-stage missing-value handling was about honesty not inventing numbers. Modeling-stage handling is stricter, because models simply cannot run on missing values at all. The features with low missingness were selected, and any row still missing a needed value was dropped. This brought the usable dataset from 226 rows down to 210 rows for regression a reasonable trade-off at this scale, and far more honest than guessing values.

```
feature_cols = ['total_tests_per_1m_population', 'population',
'testing_intensity'] + continent_dummies.columns.tolist()
target_col = 'total_deaths_per_1m_population'

model_df = df_m[feature_cols + [target_col]].dropna()
print(f"Rows available for modeling: {len(model_df)} (out of {len(df_m)})")
model_df.head()

✓ 0.0s Python
```

Rows available for modeling: 210 (out of 226)

	total_tests_per_1m_population	population	testing_intensity	continent_Asia	continent_Au
0	23455.0	40560636	0.023455	True	
1	632857.0	2871945	0.632857	False	
2	5093.0	45325517	0.005093	False	
3	3223924.0	77495	3.223924	False	
4	43136.0	34769277	0.043136	False	

Modeling and Algorithm Description

Part A: Linear Regression (supervised)

Task: Predict total_deaths_per_1m_population (deaths per million people) from testing rate, population, testing intensity, and continent.

Linear Regression fits a straight-line relationship between the input features and the numeric target:

$\text{deaths_per_1m} \approx b_0 + b_1 \cdot (\text{tests_per_1m}) + b_2 \cdot (\text{population}) + b_3 \cdot (\text{testing_intensity}) + \text{continent effects}$

It finds the coefficients that make the predicted line sit as close as possible, on average, to every real data point technically, it minimizes the sum of squared errors. Linear Regression was chosen as a first model because it's simple, interpretable (each coefficient shows how much the prediction changes per unit of that feature), and gives a fair baseline. With only about 200 rows and a handful of features, a more complex model (like Random Forest) risks memorizing noise instead of learning a real pattern so the plan was to start simple and only add complexity if this clearly underperformed.

Before training, numeric features were scaled (StandardScaler) so that population which is in the hundreds of millions and testing_intensity which sits between 0 and a few could be compared fairly. The scaler was fit only on the training data and then applied to the test data, to avoid leaking test-set information into training.

```
X = model_df[feature_cols]
y = model_df[target_col]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
num_to_scale = ['total_tests_per_1m_population', 'population', 'testing_intensity']
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()
X_train_scaled[num_to_scale] = scaler.fit_transform(X_train[num_to_scale])
X_test_scaled[num_to_scale] = scaler.transform(X_test[num_to_scale])

print(f"Train rows: {len(X_train)}, Test rows: {len(X_test)}")
```

✓ 0.0s

Python

Train rows: 168, Test rows: 42

```
reg = LinearRegression()
reg.fit(X_train_scaled, y_train)

coef_table = pd.Series(reg.coef_, index=X.columns).sort_values(key=abs, ascending=False)
coef_table
```

✓ 0.0s

Python

```
continent_Europe      2226.834625
continent_South America 1840.452994
continent_North America 1015.195945
continent_Asia         380.868752
continent_Australia/Oceania -72.175787
total_tests_per_1m_population -37.691053
testing_intensity      -37.690951
population             -28.792580
dtype: float64
```

Part B: K-Means Clustering (unsupervised)

Task: Group countries into behavioral 'profiles' using case rate, death rate, and testing rate per 1M no target variable involved.

K-Means groups similar rows together by: (1) placing k random cluster centers, (2) assigning every country to its nearest center, (3) moving each center to the average position of its assigned countries, and (4) repeating until the assignments stop changing. K-Means was chosen because it's the standard first clustering algorithm fast, easy to interpret, and a reasonable starting assumption when clusters are expected to be roughly round and evenly sized.

```
cluster_features = ['total_cases_per_1m_population', 'total_deaths_per_1m_population',
                    'total_tests_per_1m_population']
cluster_df = df_m[['country', 'continent'] + cluster_features].dropna()

scaler2 = StandardScaler()
X_cluster = scaler2.fit_transform(cluster_df[cluster_features])
print(f"Rows available for clustering: {len(cluster_df)}")
```

✓ 0.0s Python

Rows available for clustering: 210

Feature scaling was mandatory here (not just nice-to-have, like in regression), because K-Means groups points by literal distance. Without scaling, a feature like total_cases_per_1m (which ranges into the hundreds of thousands) would completely dominate the distance calculation over a feature ranging 0–10,000.

The number of clusters (k) wasn't guessed it was chosen using the elbow method (how much 'within-cluster spread' drops as, k increases) together with the silhouette score (how well-separated the clusters are, from -1 to 1).



Both plots pointed to k = 3 as a good balance a clear bend in the elbow curve and a solid silhouette score at that point.

Results with appropriate metrics

Regression results:

```
y_pred = reg.predict(X_test_scaled)

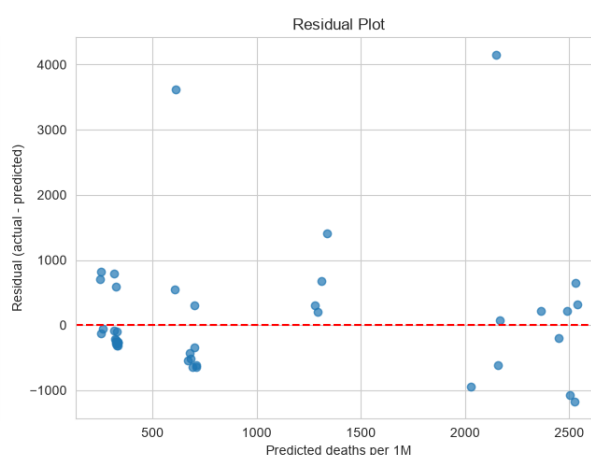
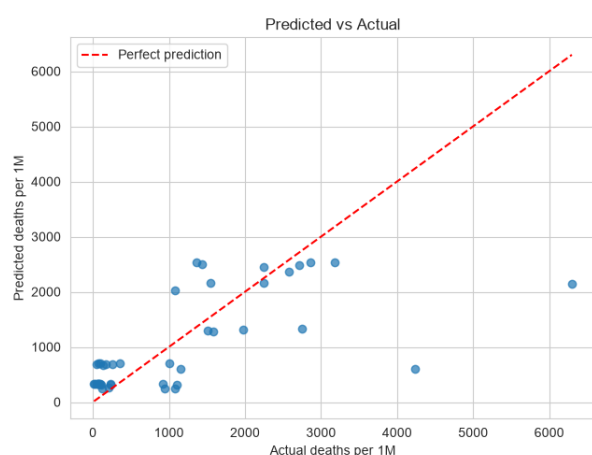
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"R^2 : {r2:.3f}")
print(f"MAE : {mae:.1f} deaths per 1M population")
print(f"RMSE : {rmse:.1f} deaths per 1M population")
```

✓ 0.0s Python

R² : 0.432
MAE : 620.8 deaths per 1M population
RMSE : 1007.3 deaths per 1M population

Metric	Value	What it means
R ² (R-squared)	0.432	The model explains about 43% of the variation in deaths per 1M. That's a modest but legitimate result for this feature set and dataset size not a failure.
MAE (Mean Absolute Error)	620.8 deaths per 1M	On average, a prediction is off by about 621 deaths per million people.
RMSE (Root Mean Squared Error)	1007.3 deaths per 1M	Since RMSE is noticeably higher than MAE, a handful of large prediction errors (likely the hardest-hit countries) are dragging this number up.



The residual plot shows errors are not perfectly random there's a tendency for the model to underpredict for countries with very high death rates, which lines up with what RMSE vs. MAE already suggested.

Which features mattered most: The continent dummy variables had by far the largest coefficients (Europe: +2,227, South America: +1,840, North America: +1,015), meaning which continent a country is in explains more of the death-rate difference than testing rate or population do in this model. Testing rate, testing intensity, and population all had small negative coefficients.

Clustering results

With $k = 3$, the clusters had a silhouette score of 0.494 a reasonably well-separated grouping for real-world data like this.

```
k_final = 3

km_final = KMeans(n_clusters=k_final, random_state=42, n_init=10)
cluster_df['cluster'] = km_final.fit_predict(X_cluster)

print(f"Final silhouette score (k={k_final}): {silhouette_score(X_cluster, cluster_df['cluster']):.3f}")
cluster_df.groupby('cluster')[cluster_features].mean().round(0)
```

✓ 0.0s Python

Final silhouette score (k=3): 0.494

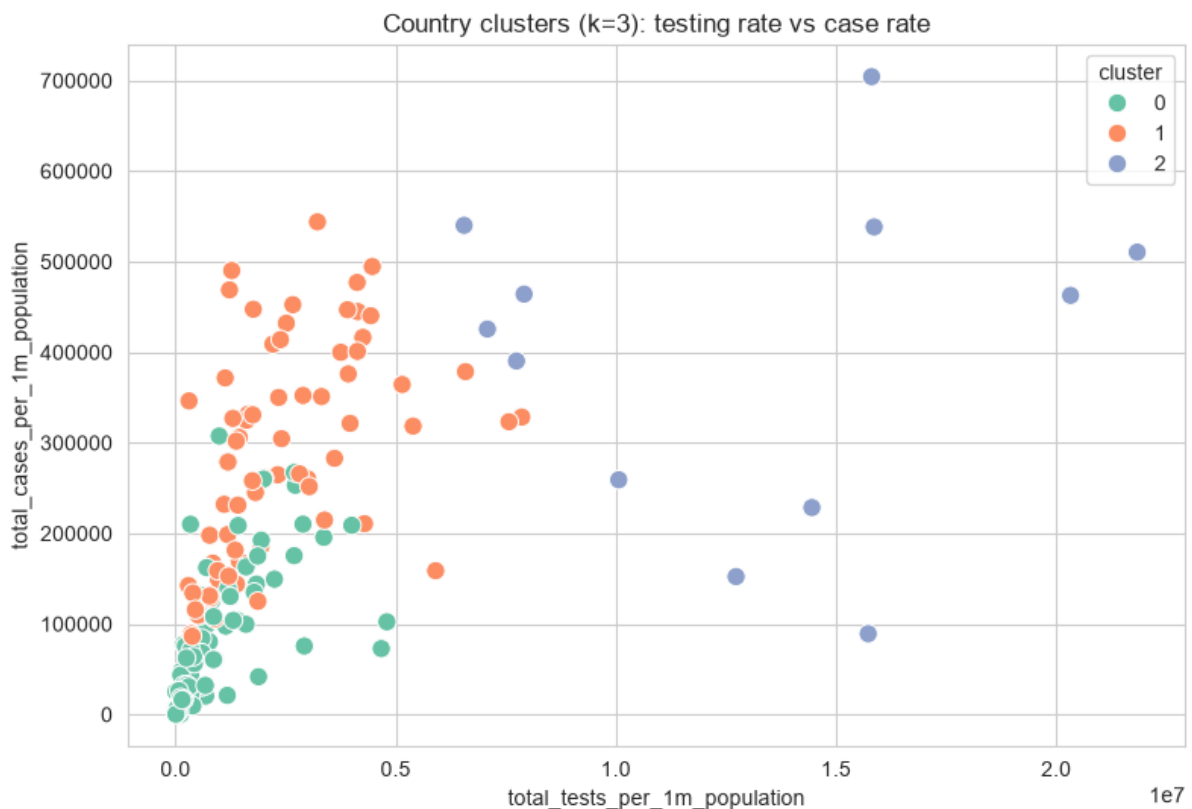
cluster	total_cases_per_1m_population	total_deaths_per_1m_population	total_tests_per_1m_population
0	58594.0	490.0	654644.0
1	281316.0	2379.0	2378940.0
2	397144.0	1253.0	13011341.0

Cluster	Avg. Cases/1M	Avg. Deaths/1M	Avg. Tests/1M	Profile
0	58,594	490	654,644	Low testing, low-to-moderate impact
1	281,316	2,379	2,378,940	High testing, high impact
2	397,144	1,253	13,011,341	Very high testing, high case rate, lower death rate

```
fig, ax = plt.subplots(figsize=(9, 6))
scatter = sns.scatterplot(data=cluster_df, x='total_tests_per_1m_population',
y='total_cases_per_1m_population',
hue='cluster', palette='Set2', s=80, ax=ax)
ax.set_title(f'Country clusters (k={k_final}): testing rate vs case rate')
plt.show()

cluster_df.groupby('cluster')['country'].apply(lambda x: ', '.join(x.head(6)) + ('...' if len(x) > 6
else ''))
```

✓ 0.1s Python



Cluster membership did not simply follow continent lines countries from several continents appear inside each cluster. This is actually the more interesting finding: it suggests testing/reporting behavior groups countries together better than geography alone does.

Discussion and conclusion

Regression

- An R^2 of 0.432 means the model captures under half of what drives death-rate differences between countries a fair, informative result, not a strong predictive model. Real-world factors this dataset doesn't capture (healthcare capacity, policy response, population density, variant timing) almost certainly explain much of the rest.
- Continent mattered far more than testing rate or population in this model, which was a bit surprising going in the assumption was that testing intensity would dominate.
- The model tends to underpredict for the worst-hit countries worth stating plainly rather than explaining over.

Clustering

- Three clusters emerged, roughly describable as: (0) low testing / lower impact, (1) high testing / high impact, and (2) very high testing / very high case rate but comparatively lower death rate.
- Clusters cut across continents rather than following them meaning testing and reporting behavior groups countries together better than geography does.

Honest limitations

- 226 rows total is small for machine learning these results are exploratory pattern-finding, not production-grade prediction.
- This is a single snapshot in time, not longitudinal data, so nothing here can speak to trends or causality only how countries compared to each other at one point in time.
- continent is a rough temporary for the real drivers of outcomes (healthcare infrastructure, policy, density), which simply aren't in this dataset.

Conclusion:

Beyond this one dataset, the transferable skills here were: the supervised-vs-unsupervised decision, reasoning through why each derived feature exists, train/test discipline (fitting the scaler on training data only), the standard regression metric trio (R^2 , MAE, RMSE), and the elbow-plus-silhouette approach to picking k for clustering. That workflow carries over to the next dataset, regardless of topic.

THE END